Unified ML Pipelines: A Reproducible Study of Training-Serving Skew, Lineage, and Release Gating

Bhavin Kotak

***Abstract*—Machine learning systems often compute features twice, once in a batch job for training and again in a per-request service for serving. Any divergence between the two programs causes training-serving skew. Version 1 of this paper argued for unified pipelines built on a single feature definition, but no releasable data stood behind its outcome figures. This version withdraws those figures and measures the central technical claim. We formalise the two feature paths, the skew and disagreement measures, a content-hash lineage record, and a fail-closed release gate. The experiment uses three public OpenML tables with five seeds. Offline, a dual implementation with five documented divergence sources matched the AUC of a single definition; online, its ROC AUC was lower by 0.086, 0.222, and 0.139. Every 95% confidence interval excluded zero, and the preregistered hypothesis was supported. Categorical encoding order caused most of the loss. The request-level skew rate was above 0.96 for every source, the harmless ones included, which makes it a weak proxy for harm. Prediction disagreement tracked harm more closely. The single definition stayed skew-free only while online-store lookups succeeded. Pinned reruns were bit-identical in 15 of 15 cases. When a rerun differed, only a lineage record with resolved seeds and parameters explained it. In a simulated release gate, adding an offline-online replay check raised the share of harmful candidates blocked from 0.522 to 0.970. It also blocked 53 of 98 harmless candidates. Code, data, and results are released so the study can be rerun.**

***Index Terms*—MLOps, training-serving skew, feature store, point-in-time correctness, data lineage, reproducibility, continuous integration, software engineering for machine learning**

I. Introduction

A machine learning (ML) model is trained on features computed by one program and is served with features computed by another. The training features usually come from a batch job over a historical table; the serving features come from an online service that answers each request under a latency budget. If the two programs disagree, the model sees inputs at serving time that differ from the ones it was trained and validated on. This failure is called training-serving skew [1], [2]. Production ML platforms such as TFX added shared transformation components partly to prevent it [3]. In interview and case studies of ML deployment, data and pipeline management keep coming up as sources of friction [4]–[7].

B. Kotak is with IBM, Armonk, NY, USA (e-mail: bhavin.kotak@ibm.com). ORCID: 0009-0006-4793-7046.

This is version 2.0 of a preprint first released in January 2026. Version 1 reported outcome figures that no data supported; this version replaces them with a preregistered experiment that anyone can rerun.

Two engineering patterns are commonly proposed in response. One is a *single feature definition*: the same feature logic runs for offline backfill and for online serving, typically through a feature store [8], [9]. The other is a unified release process. Feature code, model code, parameters, and deployment configuration are versioned together, and a change is promoted only through automated checks [10], [11]. Both patterns are widely recommended. Few studies measure their effect in a way that others can rerun.

Version 1 of this paper described such an architecture and reported large organisational gains, for example shorter model-to-production time and fewer skew incidents. There was no data behind those figures that could be released or checked, so this version removes them. The architecture and the patterns stay. We state them formally and measure the central technical claim with a preregistered experiment on public data.

We ask five research questions.

- RQ1: Does a single feature definition give higher online discrimination (ROC AUC) than a dual implementation that contains documented, realistic divergences?
- RQ2: Which divergence sources cause the harm, and is the request-level skew rate a usable proxy for that harm?
- RQ3: Where does the single-definition pattern itself break (store timeouts, heavier missingness, covariate shift)?
- RQ4: What must a lineage record contain so that a rerun is either bit-identical or has its difference explained?
- RQ5: Does a release gate with a skew replay check block injected regressions that an accuracy-only gate promotes?

The paper makes four contributions. The first is a formal system model of the two feature paths, the skew and disagreement measures, a content-hash lineage record, and a fail-closed release gate. It comes with a reference architecture that states how every external decision point behaves under missing data, unknown results, and timeouts (Section III). The second is an open experiment harness on three OpenML tables with five seeds, with a per-source ablation, three breakpoint scenarios, a rerun study, and a simulated gate (Section IV). Third, we give measured answers to RQ1 to RQ5, negative and mixed results included. Fourth, we list the version 1 claims that were removed or reframed (Section V).

II. Related Work

A. Software engineering for ML

Amershi et al. describe a nine-stage ML workflow at a large software company and the data-management challenges

that distinguish it from conventional software [4]. Paleyes et al. survey published deployment case studies and organise the challenges by workflow stage [5], and Lwakatare et al. review industrial ML systems and their challenges [7]. In interviews by Shankar et al., practitioners report that data and pipeline issues dominate operational work [6]. Kreuzberger et al. define MLOps and its components from literature and interviews [10]. These works motivate our questions. None of them measures skew under controlled conditions.

B. Platforms and feature stores

TFX puts data validation, transformation, training, and serving in one platform [3]. Orr et al. discuss feature stores as the system that manages feature definitions for both training and serving [8]. Feast is an open-source feature store with offline and online stores [9]. Xin et al. analyse a large corpus of production pipelines and their redundant computation [12]. We do not evaluate any of these systems. What we implement is the pattern they share, one definition executed in both contexts, and we compare it with a dual implementation.

C. Data validation and testing

The ML Test Score lists tests for production readiness, among them checks that training and serving features agree [2]. Polyzotis et al. survey data lifecycle challenges, training-serving skew among them [1]. Schelter et al. automate data quality verification with declarative constraints [13], and Zhang et al. survey ML testing [14]. For continuous integration of models, Renggli et al. give statistically grounded conditions [15]. Kaufman et al. formalise leakage; point-in-time violations in aggregate features are one instance of it [16]. Gama et al. survey drift and shift detection, and Rabanser et al. study them empirically [17], [18]. Our gate combines an accuracy check of the kind studied in [15] with a direct offline-online replay. We measure what each of the two catches.

D. Provenance and reproducibility

Simmhan et al. survey data provenance and classify systems by what they record and how [19]. Vamsa infers provenance from data science scripts [20]. Schlegel and Sattler survey ML artefact management [21]. Reproducibility gaps in ML research are documented by Gundersen and Kjensmo and by Pineau et al. [22], [23]. Pham et al. measure the variance that nondeterminism adds to deep learning training [24]. Our lineage study is narrower. It asks which fields a content-hash record must include so that a rerun of a tabular model is either bit-identical or flagged.

III. Methodology

A. System model

A stream of events $r_t = (x_t, g_t, y_t)$, $t = 1, \dots, N$, arrives in time order. Here x_t holds numeric attributes x_t^{num} and categorical attributes x_t^{cat}, g_t is an entity key (one categorical attribute), and $y_t \in \{0, 1\}$ is a label revealed after the prediction for t is made. The history before t is $H_{<t} = \{r_s : s < t\}$. Indices $t \le n$ form the training window; indices $t > n$ form the serving window T. A feature definition is a map $\phi(x_t, H_{<t}) \in \mathbb{R}^d$. It returns imputed numeric attributes, one derived ratio $x^{(a)}/(x^{(b)} + 1)$, an integer code for each categorical attribute, and three entity aggregates. Let $\mathbf{1}[\cdot]$ be the indicator function, v_s the numeric attribute being aggregated, m_v its training median, π the training positive rate, and $\alpha > 0$ a smoothing pseudo-count. The point-in-time aggregates for entity $g = g_t$ are

$$\begin{aligned} c_g(t) &= \textstyle\sum_{s<t} \mathbf{1}[g_s = g], \\ \bar{v}_g(t) &= \frac{\sum_{s<t} \mathbf{1}[g_s = g]\, v_s + \alpha m_v}{c_g(t) + \alpha}, \\ \rho_g(t) &= \frac{\sum_{s<t} \mathbf{1}[g_s = g]\, y_s + \alpha \pi}{c_g(t) + \alpha}. \end{aligned} \tag{1}$$

The sums in (1) range over $s < t$ only. That is the point-in-time constraint: a feature for event t may not depend on event t or on any later event [16]. For an entity with no history, $c_g(t) = 0$ and the aggregates equal their priors m_v and π.

TABLE I
Divergence Sources of the Dual Implementation

Source	Batch (training)	Per request (serving)
missing default	missing numeric field → training median	missing numeric field → 0
precision	float64, no rounding	float32 inputs; ratio and aggregates rounded to 2 decimals
encoding order	sorted vocabulary; unknown → −1	vocabulary in first-seen file order; unknown → 0
stale window	aggregates up to $t-1$	aggregates from a snapshot refreshed every 1000 requests
PIT leakage	aggregates over the whole table, including t and later rows	aggregates up to $t-1$

B. Two feature paths

Every system computes an offline feature vector $\phi^{\text{off}}(t)$ for training and offline validation, and an online feature vector $\phi^{\text{on}}(t)$ for serving. The *single-definition path* executes the same function in both contexts. Offline backfill replays history through ϕ, and serving calls ϕ against an online store that holds the running sums of (1). The *dual path* is two programs: a vectorised batch job ϕ_θ^B for training, and a per-request function ϕ_θ^S, written independently, for serving. The vector $\theta \in \{0, 1\}^5$ switches five divergence sources on or off:

$$\left(\phi^{\text{off}}, \phi^{\text{on}}\right) = \begin{cases} (\phi,\ \phi) & \text{single definition,} \\ (\phi_\theta^B,\ \phi_\theta^S) & \text{dual implementation.} \end{cases} \tag{2}$$

Table I defines the sources of (2). Each one mirrors a class of divergence discussed in prior work and in version 1 of this paper: imputation defaults, numeric precision in serialisation, categorical vocabulary order, aggregate freshness, and point-in-time leakage [1], [2], [16]. With $\theta = \mathbf{0}$ the dual code must reproduce ϕ exactly. We test this as a correctness control.

C. Measures and objective

For a request $t \in T$ and feature j, a mismatch is recorded when the two paths differ by more than a tolerance $\epsilon = 10^{-9}$ in absolute and relative terms. The request-level skew rate S counts requests with at least one mismatched feature:

$$m_{tj} = \mathbf{1}\big[\,|\phi_j^{\text{off}}(t) - \phi_j^{\text{on}}(t)| > \epsilon\,(1 + |\phi_j^{\text{off}}(t)|)\,\big]\,, \qquad S = \frac{1}{|T|}\sum_{t\in T} \max_j m_{tj}. \tag{3}$$

We also report the cell mismatch rate $\frac{1}{|T|d}\sum_{t,j} m_{tj}$. A mismatch does harm only if it changes a decision. For a fitted scorer f and threshold $\tau = 0.5$, the prediction disagreement rate is

$$D = \frac{1}{|T|}\sum_{t\in T} \mathbf{1}\big[\mathbf{1}[f(\phi^{\text{off}}(t)) \geq \tau] \neq \mathbf{1}[f(\phi^{\text{on}}(t)) \geq \tau]\big]\,. \tag{4}$$

Let f_1 be trained on $\{\phi(t)\}_{t\leq n}$ and f_2 on $\{\phi_\theta^B(t)\}_{t\leq n}$. The primary endpoint is the paired difference in online ROC AUC on the serving window,

$$\Delta = \mathrm{AUC}(\{f_1(\phi(t)), y_t\}_{t\in T}) - \mathrm{AUC}\big(\{f_2(\phi_\theta^S(t)), y_t\}_{t\in T}\big)\,. \tag{5}$$

A positive Δ means the single definition serves a better ranking than the dual implementation. The offline-online gap of a path is its offline AUC minus its online AUC. We stated the preregistered decision rule (Section IV-A) on Δ. S is only a proxy, and the rule does not use it.

D. Lineage record

Let $H(\cdot)$ be SHA-256 over a canonical serialisation. For a training run with feature matrix F, labels y, code C (feature definition and model constructor source), parameters P, and environment E (library versions), the lineage identifier is

$$\ell = H(H_{\text{data}}(F, y) \,\|\, H(C) \,\|\, H(P) \,\|\, H(E))\,. \tag{6}$$

We compare two ways of filling in this record. The *explicit* record hashes the data as an unordered set of rows, and hashes P as the user-written parameter file, in which an unset seed is stored as `None`. The *resolved* record hashes the rows in training order and stores library versions plus every constructor parameter after resolution, the concrete seed included. A rerun is bit-identical when the SHA-256 of its held-out predicted probabilities equals that of the reference run. A non-identical rerun counts as *explained* if its ℓ differs from the reference.

E. Release gate

A candidate pipeline is promoted only if three checks pass on a validation window $V \subset T$ (the first half of the serving window). Let L be true when every lineage field is present, A_c and A_0 the offline AUC of the candidate and the incumbent on V, $\delta = 0.005$ the AUC margin, S_c the candidate's skew rate (3) from replaying V through its offline and online paths, and $s_{\max} = 0.01$ the skew limit:

$$\text{promote} = L \wedge (A_c \geq A_0 - \delta) \wedge (S_c \leq s_{\max})\,. \tag{7}$$

A conjunct is false when its check raises an error, times out, or returns an undefined value, such as an AUC on a single-class window. The gate therefore fails closed. The *AUC-only* gate is the same rule without the last conjunct. We label a candidate *harmful* if its online AUC on the post-gate window $T \setminus V$ is more than 0.005 below the incumbent's.

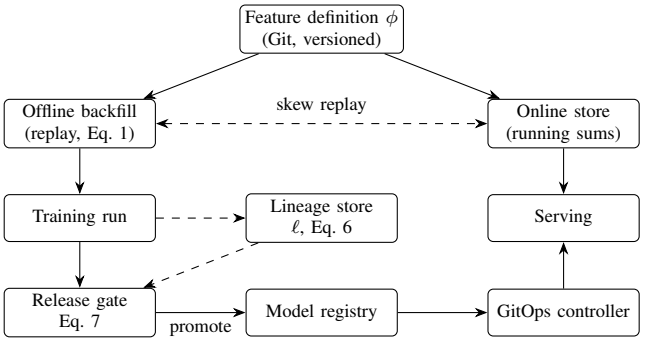


Fig. 1. Reference architecture. One feature definition feeds both offline backfill and online serving. Dashed arrows carry lineage and the offline-online replay that the release gate uses. Labels refer to Eqs. (1), (6), and (7).

F. Reference architecture and outcome handling

Fig. 1 shows the architecture kept from version 1, cut down to the parts the experiment exercises. Feature definitions, model code, and deployment manifests live in one Git repository. The same definition feeds an offline backfill and an online store. Every training run writes the lineage record (6). The gate (7) decides promotion to a model registry, and a GitOps controller reconciles serving with the registry [11]. Table II states how each external decision point behaves when data are missing, a result is unknown, or a call times out. The released code implements these rules. Its unit tests also exercise the rules for the store, the feature definition, the gate, and the registry.

IV. Evaluation

A. Setup

1) Data: Table III lists three public tables retrieved from OpenML [25]: Adult [26], Bank Marketing [27], and German Credit (credit-g) [28]. We drop the Bank Marketing call-duration column, since its value is known only after the outcome. Each file is cached with its SHA-256 checksum, and a mismatch aborts the run (Table II). The entity keys are occupation (Adult), job (Bank Marketing), and purpose (credit-g), and the aggregated numeric attributes are hours per week, account balance, and credit amount, in the same order. The smoothing pseudo-count in (1) is $\alpha = 20$.

2) Event stream: The tables carry no timestamps, so time is simulated. For each seed we draw a random arrival order. The first 70% of arrivals form the training window, and the last 30% are replayed one by one as serving requests. Each label is revealed as soon as its request has been scored. To exercise imputation, we inject missing-completely-at-random values into 5% of numeric cells of every table (a simulation). Adult's own missing categories are left in place.

TABLE II
BEHAVIOUR OF EXTERNAL DECISION POINTS UNDER MISSING DATA, UNKNOWN RESULTS, AND TIMEOUTS

Decision point	Missing data	Unknown result	Timeout
Online store lookup	Unknown entity: count 0, aggregates equal priors m_v, π	Not applicable (lookup returns a tuple or fails)	Serve fallback (count 0, priors); mark request degraded; count in S
Feature definition	Missing numeric: training median; missing or unseen category: code -1	Not applicable (pure function)	Not applicable (in process)
Release gate check	Missing lineage field: block	Error, NaN, or single-class AUC: block (fail closed)	Treated as unknown: block
Model registry	Requested version absent: keep incumbent	Artefact hash differs from lineage: refuse to load, keep incumbent	Keep incumbent; retry on next reconcile
Data download (OpenML)	Not applicable (cached file used)	sha256 differs from manifest: abort run	Retry with alternate parser, then abort with error

TABLE III
DATASETS AND SERVING WINDOWS (SEED 0)

Dataset	Rows	Num.	Cat.	Pos. rate	Train	Requests
adult	48842	6	8	0.239	34189	14653
bank-marketing	45211	6	9	0.117	31647	13564
credit-g	1000	7	13	0.300	700	300

3) Model: All paths use scikit-learn's histogram gradient boosting classifier [29] with 150 iterations, learning rate 0.1, feature subsampling 0.8, and no early stopping. Categorical codes go in as ordinal numeric inputs. Each model fit uses one OpenMP thread.

4) Protocol: Each dataset is run with five seeds (0 to 4). A seed fixes the arrival order, the injected missingness, and the model seed. For every dataset and seed we run the single path, the dual path with all five sources, the dual control with no source, five single-source ablations, and the single path with online-store timeout probability 0.01, 0.05, and 0.2. On a timeout the store serves the fallback of Table II. Two further breakpoint scenarios rerun the single and dual paths. In the first, missingness rises to 20% of numeric cells, four times the base rate. The second is covariate shift: arrivals are ordered by age plus Gaussian noise whose standard deviation is 0.5 times that of age, so the serving window holds older people.

5) Statistics and preregistration: We report means and standard deviations over the five seeds. Paired differences get two-sided 95% t confidence intervals (4 degrees of freedom). Before the full run we registered the hypothesis, the endpoint Δ of (5), and the rule (file `preregistration.json`): the hypothesis is SUPPORTED if, in at least two of three datasets, the mean $\Delta \geq 0.005$ and the lower confidence bound exceeds 0; REFUTED if, in at least two of three datasets, the upper bound is below 0.005; and INCONCLUSIVE otherwise. The code computes the verdict from the result files.

6) Reproducibility study: On the seed-0 split of each dataset, we retrain the single-path model five times under each of five regimes. They are: pinned (fixed seed, data, parameters); pinned with one versus two OpenMP threads; unpinned seed (fresh operating-system entropy, with the parameter file recording no seed); training rows in a different order with the seed pinned; and a simulated library default change (L2 regularisation 0 versus 1, not recorded in the parameter file). Each regime has its own reference run.

TABLE IV
PRIMARY ENDPOINT: OFFLINE AND ONLINE ROC AUC (MEAN $\pm$ SD OVER FIVE SEEDS) AND PAIRED Δ WITH 95% CI

Dataset	Path	Offline AUC	Online AUC	Gap
adult	single	0.922±0.002	0.922±0.002	0.000
	dual	0.924±0.002	0.836±0.019	0.088
	Δ	0.086 [0.063, 0.109]		
bank-marketing	single	0.783±0.005	0.783±0.005	0.000
	dual	0.794±0.003	0.560±0.040	0.234
	Δ	0.222 [0.171, 0.274]		
credit-g	single	0.763±0.032	0.763±0.032	0.000
	dual	0.775±0.013	0.625±0.071	0.150
	Δ	0.139 [0.072, 0.205]		

7) Gate simulation: For each dataset and seed, the incumbent is the pipeline with all sources off, and eleven candidates go through (7). Three of them are benign: a new model seed (seed plus 1000), learning rate 0.09, and shuffled training rows. The other eight are injected regressions (simulations): 20% of training labels flipped; one column dropped upstream (Adult capital gain, Bank Marketing previous outcome, credit-g checking status); an underfit model with 5 iterations; point-in-time leakage in the training batch job; and a serving-only bug for each of the other four sources.

B. Results

1) Correctness control: With every source switched off, the dual code reproduced the single definition exactly. The maximum absolute feature difference was 0 for all three datasets and five seeds, in both the training and the serving window (file `controls.csv`). Every difference reported below is therefore caused by the switched sources.

2) Online discrimination (RQ1): Table IV and Fig. 2 give the primary endpoint. On offline validation the two paths look alike; on Adult the dual path scored 0.924 and the single path 0.922. Online, the dual path fell to 0.836, 0.560, and 0.625 on Adult, Bank Marketing, and credit-g, while the single path served exactly its offline AUC. The paired difference Δ of (5) was 0.086 [0.063, 0.109] on Adult, 0.222 [0.171, 0.274] on Bank Marketing, and 0.139 [0.072, 0.205] on credit-g. All three datasets met the preregistered support condition, so

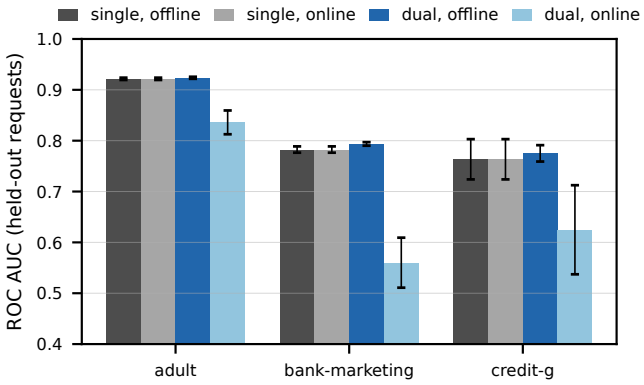


Fig. 2. Offline and online ROC AUC on the serving window for the single definition and the dual implementation with all five sources (mean over five seeds; error bars show 95% CI half-widths).

TABLE V
SINGLE-SOURCE ABLATION (MEANS OVER FIVE SEEDS). REQ. SKEW IS S OF (3); DISAGREE IS D OF (4)

Dataset	Source	Req. skew	Cell mism.	Disagree	ΔAUC
adult	missing default	1.000	0.067	0.034	0.025
	precision	1.000	0.115	0.004	0.000
	encoding order	1.000	0.418	0.119	0.062
	stale window	0.985	0.164	0.003	0.000
	PIT leakage	1.000	0.167	0.002	-0.002
	all five	1.000	0.599	0.129	0.086
bank-marketing	missing default	1.000	0.071	0.006	0.004
	precision	1.000	0.120	0.007	0.011
	encoding order	1.000	0.330	0.082	0.226
	stale window	0.988	0.156	0.001	0.000
	PIT leakage	1.000	0.158	0.002	-0.011
	all five	1.000	0.518	0.078	0.222
credit-g	missing default	0.965	0.059	0.059	0.006
	precision	0.999	0.110	0.009	0.001
	encoding order	1.000	0.466	0.245	0.151
	stale window	0.967	0.121	0.046	-0.001
	PIT leakage	1.000	0.125	0.017	-0.010
	all five	1.000	0.636	0.253	0.139

the verdict computed by the code is SUPPORTED. For the dual path, the offline-online gap was 0.088, 0.234, and 0.150, respectively. Offline validation alone did not reveal it.

3) Which sources cause harm (RQ2): Table V and Fig. 3 attribute the effect to sources. Categorical encoding order alone produced most of it, with Δ of 0.062 on Adult, 0.226 on Bank Marketing, and 0.151 on credit-g. On Adult the second source was the missing-value default, at 0.025 [0.022, 0.029]. On Bank Marketing that source gave 0.004 [0.002, 0.006], and on credit-g its interval included 0. Precision rounding and the stale window changed online AUC by at most 0.011 in absolute value, and all of their intervals included 0. Point-in-time leakage gave a negative Δ on Adult (-0.002) and Bank Marketing (-0.011), so the leaky model served slightly better rankings. Our entity keys have between 10 and 14 values. Each whole-table aggregate therefore averages thousands of rows, and the leaked label carries little weight. Leakage through high-cardinality keys, the case discussed in [16], was not tested.

The request-level skew rate did not separate harmful from harmless sources. Every source produced $S \geq 0.96$, the harmless ones too, because once any input to an aggregate feature differs, that feature differs on almost every request. With all sources on, all 73265 Adult requests (five seeds) were skewed, while 9438 of them (0.129) changed their thresholded prediction. The prediction disagreement rate D tracked harm more closely. On every dataset the two sources with the largest D were also the two with the largest Δ, and encoding order ranked first on both. The cell mismatch rate did not track harm: point-in-time leakage touched 0.167 of Adult feature cells, more than the missing default (0.067), and still caused no loss. A skew monitor that alarms on S alone would alarm on every source. One that alarms on D would do so roughly in proportion to harm.

4) Where the single definition breaks (RQ3): The single definition is free of skew only as long as every store lookup succeeds. In Fig. 4 the skew rate tracked the timeout probability: on Adult it was 0.0097, 0.0504, and 0.1995 for timeout probabilities 0.01, 0.05, and 0.2. Every degraded request was skewed and no other request was (14619 of 73265 Adult requests at probability 0.2). The degraded flag of Table II thus identifies skewed requests exactly. The harm was small, because the fallback replaces only the three aggregates. At probability 0.2 the online AUC loss was 0.0012 [0.0004, 0.0020] on Adult, and the intervals for Bank Marketing and credit-g included 0.

TABLE VI
BREAKPOINT SCENARIOS: ONLINE AUC (MEAN OVER FIVE SEEDS) AND PAIRED Δ WITH 95% CI

Dataset	Scenario	Single	Dual	Δ [95% CI]
adult	base	0.922	0.836	0.086 [0.063, 0.109]
	4x missing	0.914	0.748	0.166 [0.128, 0.204]
	shift	0.886	0.823	0.063 [0.054, 0.073]
bank-marketing	base	0.783	0.560	0.222 [0.171, 0.274]
	4x missing	0.781	0.541	0.240 [0.189, 0.291]
	shift	0.784	0.549	0.235 [0.195, 0.276]
credit-g	base	0.763	0.625	0.139 [0.072, 0.205]
	4x missing	0.753	0.612	0.141 [0.095, 0.188]
	shift	0.748	0.628	0.120 [0.011, 0.229]

Table VI covers the other two breakpoints. Raising missingness to 20% of numeric cells nearly doubled Δ on Adult, from 0.086 to 0.166 [0.128, 0.204], because the zero fill of the serving path now fires four times as often. The single path's online AUC was 0.914 here, against 0.922 at the base rate. Under covariate shift by age, Δ stayed positive on all three datasets: 0.063 [0.054, 0.073], 0.235 [0.195, 0.276], and 0.120 [0.011, 0.229].

5) What lineage must record (RQ4): Table VII summarises the five regimes. With seed, data, and parameters pinned, all 15 reruns produced bit-identical held-out predictions, and so did the reruns that alternated between one and two OpenMP threads. With the seed left unset, none of the 15 reruns matched its reference, and predicted probabilities moved by up to 0.465. The explicit record stores the seed as unset. It changed in none of these 15 cases and so could not explain the difference, while the resolved record, which stores the

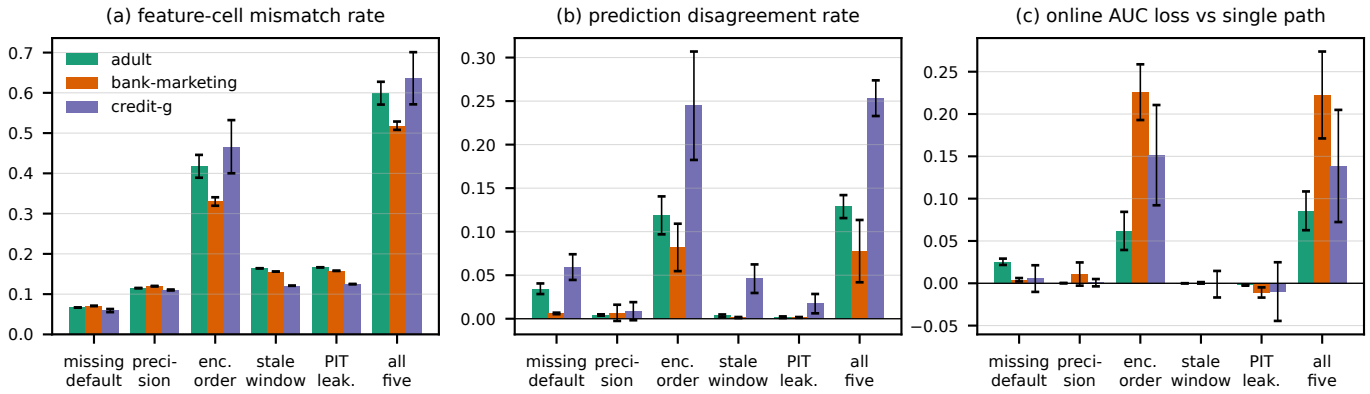


Fig. 3. Single-source ablation of the dual implementation. (a) Fraction of feature cells that differ between offline and online paths. (b) Fraction of requests whose thresholded prediction flips. (c) Online AUC loss relative to the single path, Δ of (5). Means over five seeds; error bars are 95% CI half-widths.

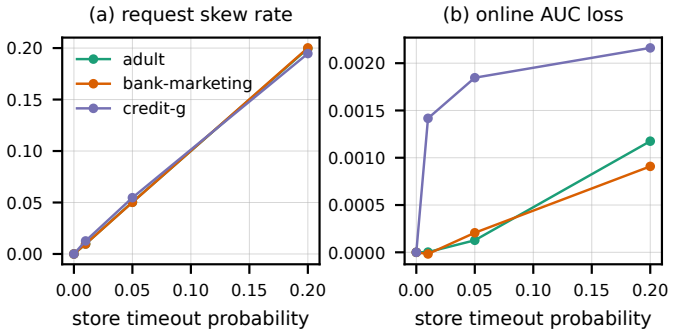


Fig. 4. Single definition with online-store timeouts. A timeout serves the documented fallback (priors). (a) Request skew rate. (b) Online AUC loss relative to the same path without timeouts. Means over five seeds.

TABLE VII
RERUN STUDY (THREE DATASETS, FIVE RERUNS EACH). IDENTICAL: BIT-IDENTICAL HELD-OUT PREDICTIONS. EXPL.: NON-IDENTICAL RERUNS WHOSE LINEAGE HASH (6) CHANGED

Regime	Identical	Differ	Expl. (explicit)	Expl. (resolved)
pinned	15/15	0	0/0	0/0
pinned, 1 vs 2 threads	15/15	0	0/0	0/0
unpinned seed	0/15	15	0/15	15/15
row order varies	15/15	0	0/0	0/0
default change (sim.)	6/15	9	0/9	9/9

concrete seed, changed in all 15. The simulated library default change behaved the same way: 9 of 15 reruns differed, the explicit record explained none of them, and the resolved record explained all 9. Reordering the training rows changed no prediction of this learner. Even so, the order-sensitive data hash of the resolved record flagged all 15 such reruns. An order-sensitive hash is thus conservative. It never missed a real change in our runs, but it raised false alarms for an order-invariant learner.

6) What the release gate catches (RQ5): Table VIII lists every candidate. All regressions in it are injected simulations. Of 165 candidate evaluations, 67 were harmful by the post-gate criterion. The AUC-only gate blocked 35 of them (0.522), and the gate with the skew replay of (7) blocked 65 (0.970). The difference came from two kinds of change that do not

TABLE VIII
SIMULATED RELEASE GATE: HARMFUL CANDIDATES AND CANDIDATES BLOCKED PER GATE, OVER 15 DATASET-SEED PAIRS. SKEW RATE IS THE MEAN S_c ON THE VALIDATION WINDOW

Candidate	Harmful	AUC-only	AUC+skew	Skew rate
seed change	0/15	2/15	2/15	0.000
learning rate 0.09	0/15	2/15	2/15	0.000
row shuffle	0/15	0/15	0/15	0.000
label noise	14/15	15/15	15/15	0.000
dropped column	13/15	13/15	13/15	0.000
underfit	9/15	11/15	11/15	0.000
training PIT leakage	1/15	2/15	15/15	1.000
serving encoding order	15/15	0/15	15/15	1.000
serving missing default	10/15	0/15	15/15	0.988
serving stale window	1/15	0/15	15/15	0.969
serving precision	4/15	0/15	15/15	1.000

lower offline AUC. Serving-only bugs do not touch the offline path, so the AUC-only gate promoted every one of them, all 15 encoding-order bugs included, though each of those was harmful. Point-in-time leakage raised offline AUC, and the AUC-only gate blocked only 2 of 15 leaky candidates. The skew replay blocked all of these.

The skew replay cost precision. The full gate blocked 53 of 98 harmless candidates (0.541), against 10 (0.102) for the AUC-only gate, because it blocked every precision and stale-window candidate when only 4 and 1 of 15 were harmful. Both gates blocked the same 4 of 45 benign changes, all on credit-g. Its validation window has 150 requests, and there the AUC margin of 0.005 is smaller than seed-to-seed variation. The two harmful candidates that passed both gates were on Bank Marketing, a dropped column and an underfit model; their AUC drop on the validation window stayed within the margin. The skew limit $s_{\max} = 0.01$ treats a harmless rounding difference the same as a harmful encoding bug. RQ2 suggests that a limit on the disagreement rate D would track harm more closely, but we did not evaluate such a gate.

V. THREATS TO VALIDITY

A. Construct validity

The author injected the divergence sources. Their magnitudes (for example, a fill value of 0, two-decimal rounding, a

refresh every 1000 requests) were fixed in the preregistration before the full run, but other choices would give other effect sizes. Readers can recombine sources from the per-source ablation. It does not estimate how often each source occurs in practice. Version 1 reported an incident taxonomy with frequencies; those frequencies had no releasable data and are not used. The primary endpoint is ROC AUC on the serving window. We report the skew rate S as a proxy, and our results show that it is a weak one.

B. Internal validity

The single path and the dual control share static constants (medians, vocabularies, priors) fitted on the training window, so the paths differ only in the switched sources. The control test confirms that, with all sources off, the dual code reproduces the single definition exactly. Labels are revealed with zero delay. In a real system with delayed labels, the label-rate aggregate would be staler for both paths. Before the preregistered full run, the code was developed with unit tests and smoke runs (seed 0 of Adult and all seeds of credit-g). The smoke runs exposed two defects, both fixed before the full run: OpenMP thread oversubscription, and the reference run of the unpinned-seed regime. The full run was started twice. We stopped the first attempt during the rerun study because of OpenMP stalls on a shared machine; its skew results were identical to those of the second. No dataset, seed, divergence parameter, or threshold was changed after either run.

C. External validity

Three tabular datasets with synthetic arrival orders are not production event streams. The tables have no timestamps, so time is simulated by random permutation, and the shift scenario by ordering on age. We use one model family, gradient-boosted trees with ordinal category codes; models with one-hot or embedding inputs react differently to encoding-order errors. The online store is an in-process dictionary, not Feast, Redis, or another networked store. Timeouts are therefore simulated by a Bernoulli draw, and latency is not measured. We did not test scale beyond a single process (two orders of magnitude more entities or requests per second), nor adversarial inputs. Feast, TFX Transform, and commercial feature stores were not available as baselines in this setting and are not compared.

D. Reproducibility study

We simulated library-version changes by changing one default parameter; no different library versions were installed. Thread-count variation covers one and two OpenMP threads on one machine, and other hardware may behave differently. The rerun study uses one split per dataset.

E. Release gate

Every regression in the gate study is an injected simulation. The harm label depends on a 0.005 AUC margin on a post-gate window, and for credit-g that window is small, so the credit-g gate outcomes are noisy. The gate study does not measure engineering cost or time to detection.

F. Removed or reframed version 1 claims

No releasable data supports the following statements of version 1, and they are withdrawn: reduced model-to-production time, reduced skew incidents, faster recovery, lower infrastructure cost, higher engineer productivity, the Netflix, bank, and healthcare case-study tables, an offline-to-production AUC drop in a financial deployment, an incident taxonomy with percentages, and claims of preventing most skew-related incidents. Statements that feature stores eliminate skew are reframed. In our setting the single definition produced no skew only while every store lookup succeeded.

VI. Conclusion

Version 1 of this paper argued for unified ML pipelines with outcome figures that nobody could check. This version keeps the architecture and states it formally. It also measures the central technical claim on public data. In our preregistered experiment, a single feature definition served better rankings than a dual implementation with five documented divergence sources on all three datasets, and offline validation of the dual path did not show the loss. Most of the loss came from one source, categorical encoding order. Two sources produced near-universal feature mismatches with no measurable harm. The request-level skew rate is therefore a weak alarm signal, and prediction disagreement is a better one. Under online-store timeouts the single definition did not remove skew. It confined skew to the requests whose lookup timed out, which the degraded flag identifies, and the resulting AUC loss was small. Pinned reruns were bit-identical, and only a lineage record that stores resolved seeds and parameters explained the reruns that were not. In the simulated gate, an offline-online replay check blocked harmful serving-side and leakage candidates that an accuracy-only gate promoted, at the price of blocking many harmless candidates.

Within the limits listed in Section V, these results support three rules for practice. Execute one feature definition in both contexts, and test the dual-path control for exact equality. Monitor the prediction disagreement between offline and online features as well as the feature mismatches. Record resolved seeds, full parameters, and ordered data hashes in the lineage record, and fail the release gate closed when any check is unknown.

Data and code are in the Zenodo package of this version: the paper source, the experiment code, the cached OpenML files with their sources, licences, and SHA-256 checksums (`data/SOURCES.md`), all result files including `verdict.json`, the unit tests, and the preregistration. Running `python code/run_all.py` regenerates every number, table, and figure, and `python -m pytest tests` runs the tests.

Acknowledgment

Generative AI was used to prepare this revision. Large language models (GitHub Copilot with Anthropic Claude) critiqued the methodology, generated the experiment code, and drafted and edited the text; in a final pass the same models also edited the wording. Every reported number, table, and

figure was produced by code from logged experiment results. None of them was written by a language model. The author is employed by IBM, and this work received no external funding.

REFERENCES

[1] N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data lifecycle challenges in production machine learning: A survey," *ACM SIGMOD Record*, vol. 47, no. 2, pp. 17–28, 2018.

[2] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *Proc. IEEE Int. Conf. Big Data*, 2017, pp. 1123–1132.

[3] D. Baylor, E. Breck, H.-T. Cheng, N. Fiedel, C. Y. Foo, Z. Haque, S. Haykal, M. Ispir, V. Jain, L. Koc, C. Y. Koo, L. Lew *et al.*, "TFX: A TensorFlow-based production-scale machine learning platform," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2017, pp. 1387–1395.

[4] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, "Software engineering for machine learning: A case study," in *Proc. IEEE/ACM 41st Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 291–300.

[5] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–29, 2022.

[6] S. Shankar, R. Garcia, J. M. Hellerstein, and A. G. Parameswaran, "Operationalizing machine learning: An interview study," arXiv:2209.09125, 2022.

[7] L. E. Lwakatare, A. Raj, I. Crnkovic, J. Bosch, and H. H. Olsson, "Large-scale machine learning systems in real-world industrial settings: A review of challenges and solutions," *Information and Software Technology*, vol. 127, p. 106368, 2020.

[8] L. Orr, A. Sanyal, X. Ling, K. Goel, and M. Leszczynski, "Managing ML pipelines: Feature stores and the coming wave of embedding ecosystems," *Proceedings of the VLDB Endowment*, vol. 14, no. 12, pp. 3178–3181, 2021.

[9] Feast Authors, "Feast: An open source feature store for machine learning," Software documentation, accessed 2026-11-04. [Online]. Available: https://docs.feast.dev

[10] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31 866–31 879, 2023.

[11] OpenGitOps Project, "GitOps principles v1.0.0," Project documentation, accessed 2026-11-04. [Online]. Available: https://opengitops.dev

[12] D. Xin, H. Miao, A. Parameswaran, and N. Polyzotis, "Production machine learning pipelines: Empirical analysis and optimization opportunities," in *Proc. Int. Conf. Management of Data (SIGMOD)*, 2021, pp. 2639–2652.

[13] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.

[14] J. M. Zhang, M. Harman, L. Ma, and Y. Liu, "Machine learning testing: Survey, landscapes and horizons," *IEEE Transactions on Software Engineering*, vol. 48, no. 1, pp. 1–36, 2022.

[15] C. Renggli, B. Karlaš, B. Ding, F. Liu, K. Schawinski, W. Wu, and C. Zhang, "Continuous integration of machine learning models with ease.ml/ci: Towards a rigorous yet practical treatment," arXiv:1903.00278, 2019.

[16] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[17] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

[18] S. Rabanser, S. Günnemann, and Z. C. Lipton, "Failing loudly: An empirical study of methods for detecting dataset shift," arXiv:1810.11953, 2018.

[19] Y. L. Simmhan, B. Plale, and D. Gannon, "A survey of data provenance in e-science," *ACM SIGMOD Record*, vol. 34, no. 3, pp. 31–36, 2005.

[20] M. H. Namaki, A. Floratou, F. Psallidas, S. Krishnan, A. Agrawal, Y. Wu, Y. Zhu, and M. Weimer, "Vamsa: Automated provenance tracking in data science scripts," in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2020, pp. 1542–1551.

[21] M. Schlegel and K.-U. Sattler, "Management of machine learning lifecycle artifacts: A survey," arXiv:2210.11831, 2022.

[22] O. E. Gundersen and S. Kjensmo, "State of the art: Reproducibility in artificial intelligence," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 32, no. 1, 2018.

[23] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," arXiv:2003.12206, 2020.

[24] H. V. Pham, S. Qian, J. Wang, T. Lutellier, J. Rosenthal, L. Tan, Y. Yu, and N. Nagappan, "Problems and opportunities in training deep learning software systems: An analysis of variance," in *Proc. 35th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2020, pp. 771–783.

[25] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," *ACM SIGKDD Explorations Newsletter*, vol. 15, no. 2, pp. 49–60, 2014.

[26] B. Becker and R. Kohavi, "Adult," UCI Machine Learning Repository, 1996. [Online]. Available: https://archive.ics.uci.edu/dataset/2/adult

[27] S. Moro, P. Cortez, and P. Rita, "A data-driven approach to predict the success of bank telemarketing," *Decision Support Systems*, vol. 62, pp. 22–31, 2014.

[28] H. Hofmann, "Statlog (German credit data)," UCI Machine Learning Repository, 1994. [Online]. Available: https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data

[29] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel *et al.*, "Scikit-learn: Machine learning in Python," arXiv:1201.0490, 2012.